■ RepoGuardian

Developer Performance Report

@codewithVamshi5

30.0

Average Health Score / 100

→ Trend: **Stable**

PRs Reviewed: **9** · Total Findings: **313** · Active Since: **2026-03-14**

Report Generated: March 14, 2026 at 23:29

1. Executive Summary

30.0	9	313	84	126
Avg Health Score	PRs Reviewed	Total Findings	Critical (High)	Warnings (Medium)

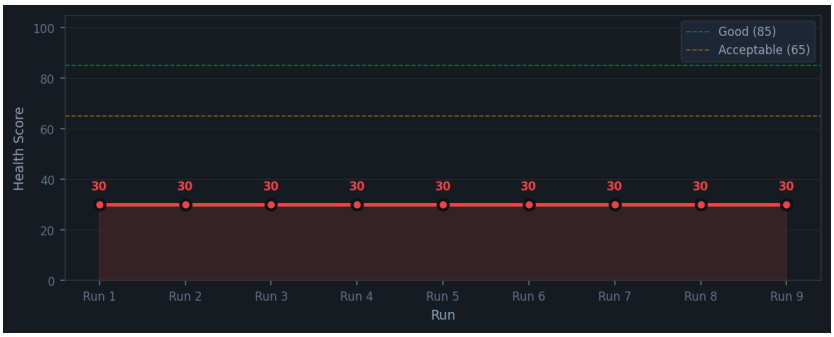
Developer **@codewithVamshi5** has been reviewed across **9 PR run(s)** since 2026-03-14, with a total of **313 findings** identified. The average health score is **30.0/100**, maintaining a consistent score — focus on addressing high-severity issues.

Critical issues requiring immediate attention: **84 high-severity**, **126 medium-severity**, and **103 low-severity** findings.

Severity Breakdown

Severity	Count	Impact	Action Required
■ High	84	Critical	Fix before merge
■ Medium	126	Warning	Fix before release
■ Low	103	Minor	Fix when possible
Total	**313**	—	—

Score Progression & Severity Distribution



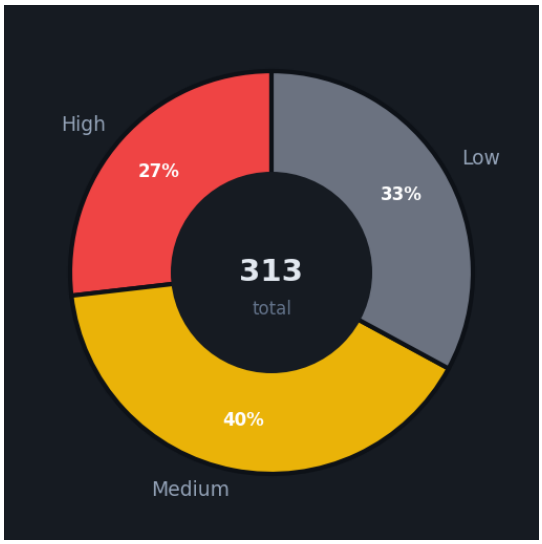


Fig 1: Health score per run · Fig 2: Finding severity distribution

2. Agent Analysis

The following section breaks down findings by agent, showing which automated agents detected the most issues for this developer.

Agent	Total	High	Medium	Low
Security	129	76	27	26
Docs	106	0	44	62
PR Review	78	8	55	15

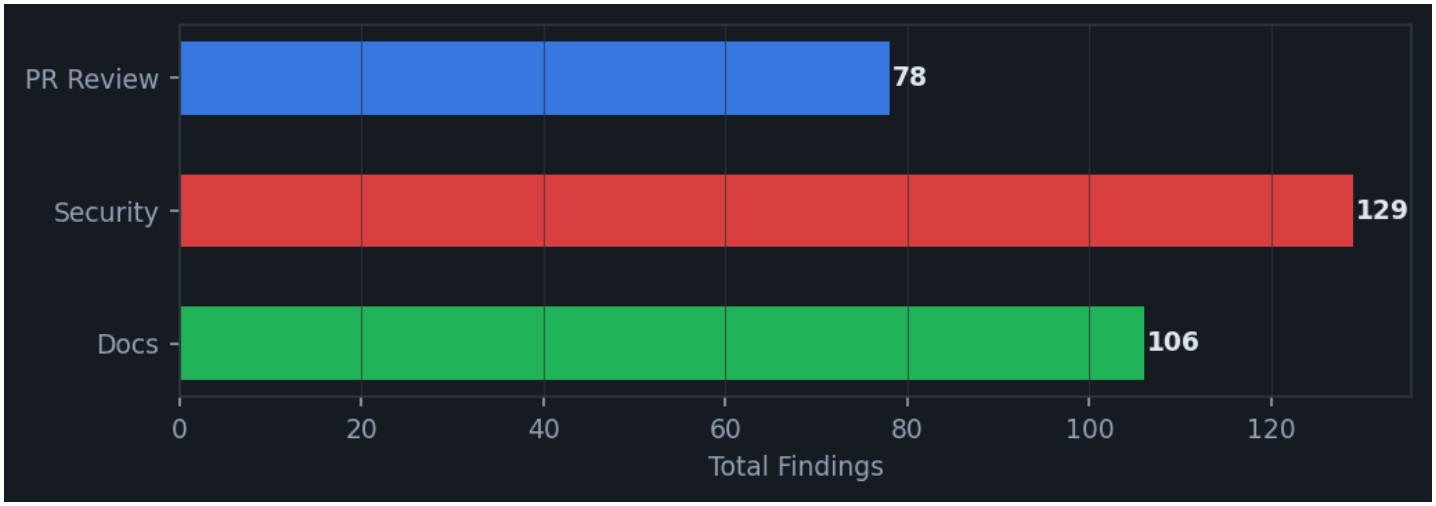


Fig 3: Total findings per agent

Issues Per Review Run

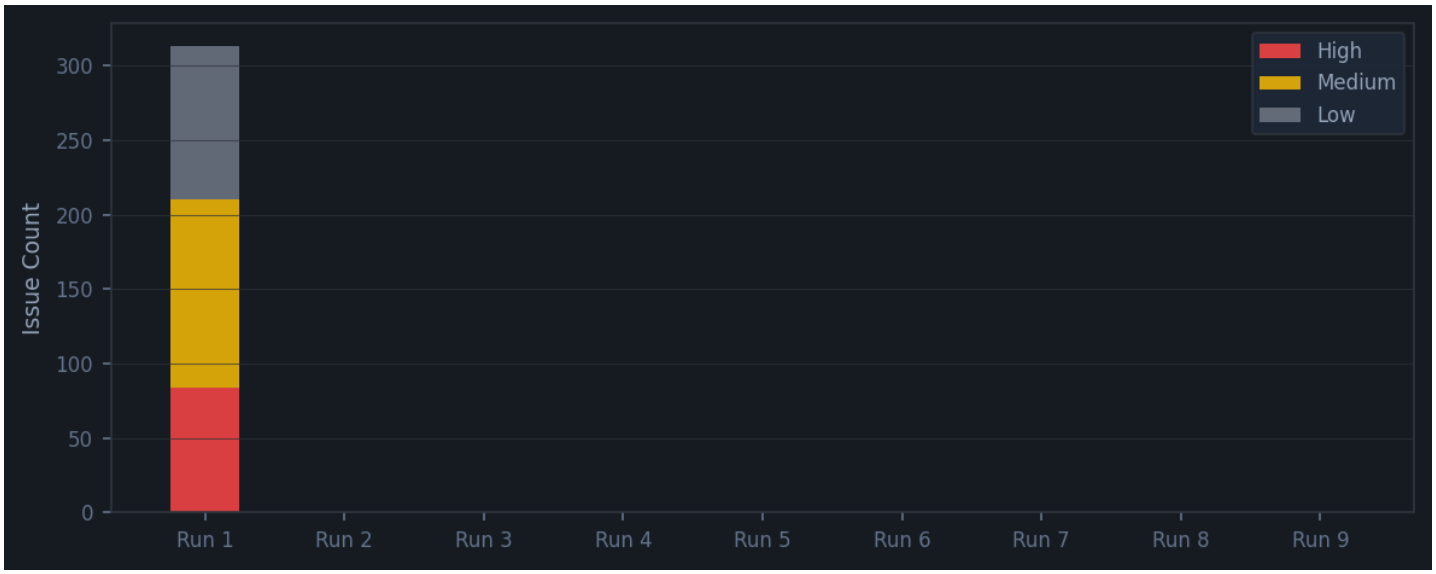
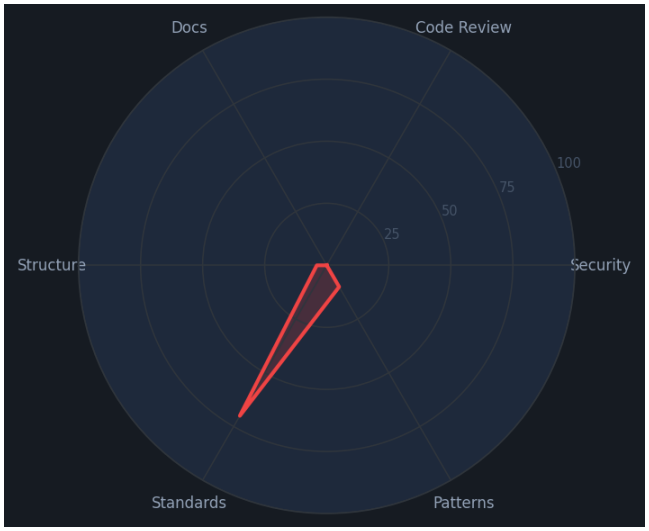


Fig 4: Issues breakdown per review run (stacked by severity)

3. Code Quality Dimensions

The radar chart below shows quality scores across six key dimensions. Each axis starts at 100 and is penalised based on findings in that category. A larger filled area indicates better overall code quality.



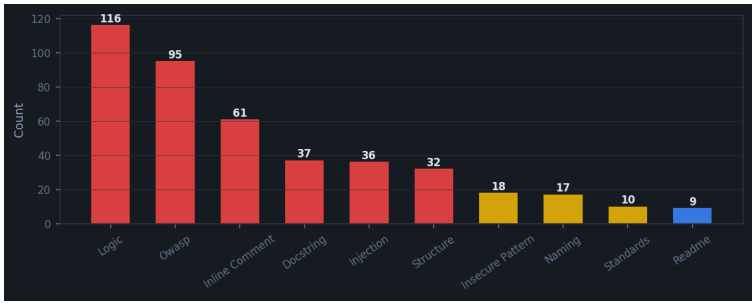


Fig 5: Code quality radar · Fig 6: Top issue categories

Recurring Issues (seen 3+ times)

These categories have appeared repeatedly across multiple PRs. Addressing these patterns will have the highest impact on code quality.

Category	Occurrences	Risk Level	Recommendation
Logic	116	High	Address in next sprint
Owasp	95	High	Address in next sprint
Inline Comment	61	High	Address in next sprint
Docstring	37	High	Address in next sprint
Injection	36	High	Address in next sprint
Structure	32	High	Address in next sprint
Insecure Pattern	18	Medium	Address in next sprint
Naming	17	Medium	Address in next sprint

4. File-Level Analysis

The following files have accumulated the most issues. Files with a high proportion of high-severity findings should be prioritised for refactoring.

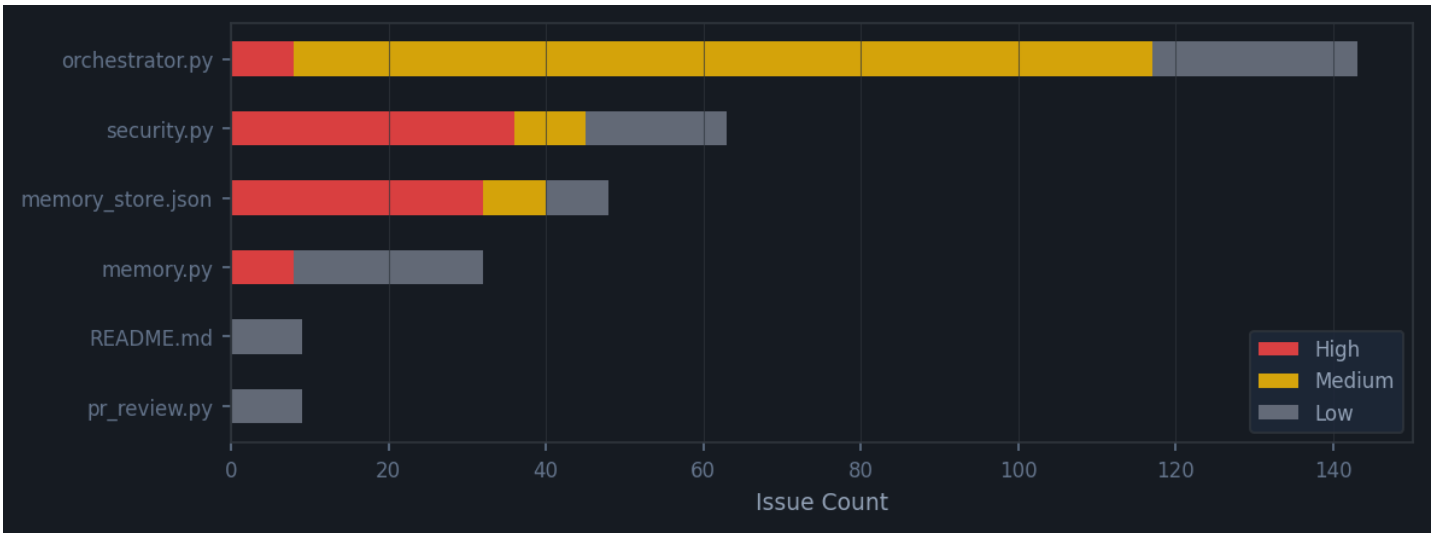


Fig 7: Top files by issue count (H/M/L breakdown)

File Details

File	Total	High	Medium	Low	Risk
orchestrator.py	143	8	109	26	High
security.py	63	36	9	18	High
memory_store.json	48	32	8	8	High
memory.py	32	8	0	24	High
README.md	9	0	0	9	Low
pr_review.py	9	0	0	9	Low

5. Critical Findings

The following 8 high-severity findings require immediate attention before any PR is approved for merge.

Severity	Agent	File : Line	Issue
HIGH	PR Review	orchestrator.py:120	The post_final_verdict function is using a try-except block to handle any exceptions that ...
HIGH	Security	memory_store.json:197	SQL injection risk: query uses string formatting or concatenation. Use parameterised queri...
HIGH	Security	memory_store.json:209	eval() executes arbitrary code — never use with user input. Use ast.literal_eval() for saf...
HIGH	Security	memory_store.json:221	Command injection risk: os.system() or subprocess shell=True. Use subprocess.run() with a ...
HIGH	Security	memory_store.json:235	pickle.load() can execute arbitrary code — never use with untrusted data. Use JSON or a sa...
HIGH	Security	memory.py:356	eval() executes arbitrary code — never use with user input. Use ast.literal_eval() for saf...
HIGH	Security	security.py:146	SQL injection risk: query uses string formatting or concatenation. Use parameterised queri...
HIGH	Security	security.py:203	eval() executes arbitrary code — never use with user input. Use ast.literal_eval() for saf...

6. Recommendations

1. Fix all **84 high-severity issues** before next PR — these block merge.
2. Address recurring pattern: **Logic** (116 occurrences) — consider a dedicated refactor sprint.
3. **Security agent** found 129 issues — prioritise a security-focused code review session.
4. Improve **documentation coverage** — add docstrings to all public functions and update README for new features.
5. Focus refactoring on **orchestrator.py** — highest concentration of issues across all runs.